

FOD FPGA Classifier — Project Recap

Checkpoint after Stage 1 and completion of the Python integer-MLP validation

1. Project Objective

The project is developing a lightweight Foreign Object Debris (FOD) classifier intended for FPGA deployment on a PYNQ-Z2 / Zynq-7020. The current approach uses RGB images, engineered image features, and a very small neural network instead of a conventional CNN. The target is a compact integer-oriented model suitable for later Vitis HLS implementation.

2. Overall Pipeline

RGB image → object segmentation / feature extraction → 64 engineered features → standardization + INT8 quantization → 64→32→16→4 MLP → class prediction → Vitis HLS → PYNQ-Z2

3. Dataset and Feature Extraction

The selected classes are fuelcap, pliers, screwdrivers, and sodacan. The project moved from grayscale processing to RGB because grayscale conversion reduced useful colour information and made some runway/background regions harder to distinguish.

Exactly 64 engineered features are extracted per image. The feature groups cover object colour, contrast relative to the background, shape, gradient, texture, and background context.

Split	Images	Feature shape
Train	1870	(1870, 64)
Validation	384	(384, 64)
Test	412	(412, 64)
Total	2666	64 features/image

4. Stage 1 — Floating-Point MLP

The final lightweight TensorFlow model is:

Input(64) → Dense(32, ReLU) → Dense(16, ReLU) → Dense(4, Softmax)

The small architecture was chosen to keep the model compact and practical for FPGA implementation.

Stage 1 test result

Metric	Result
Test accuracy	98.30%
Test loss	0.09787
Test samples	412

5. Stage 1 Confusion Matrix

On the 412-image test set, pliers, screwdrivers, and sodacan had no classification errors. The remaining errors were concentrated in fuelcap.

True \ Pred.	Fuelcap	Pliers	Screwdrivers	Sodacan
Fuelcap	55	0	6	1
Pliers	0	126	0	0
Screwdrivers	0	0	97	0

Sodacan	0	0	0	127
----------------	---	---	---	-----

6. Input Quantization

The standardized feature vector is clipped to ± 4 and mapped to signed INT8:

```
x_std = (x - mean) / std
x_clip = clip(x_std, -4, +4)
x_int8 = round(x_clip * 127 / 4)
```

The input scale is $S_X = 4/127 = 0.03149606299$. The saved INT8 training data was reconstructed exactly: maximum difference 0, mean absolute difference 0, and zero differing values.

7. Weight and Bias Quantization

Each weight matrix uses symmetric per-layer INT8 quantization: $S_W = \max(|W|)/127$ and $W_{\text{int8}} = \text{round}(W/S_W)$. Biases are represented as INT32 values in the corresponding accumulator scale.

Layer	Weight shape	Weight scale
W1	(64, 32)	0.0025182366
W2	(32, 16)	0.0033579213
W3	(16, 4)	0.004470473

8. Integer MLP Architecture

The validated integer computation is:

```
INT8 input (64)
↓
INT8 × INT8 MAC → INT32 + bias → ReLU → INT8 (32)
↓
INT8 × INT8 MAC → INT32 + bias → ReLU → INT8 (16)
↓
INT8 × INT8 MAC → INT32 logits (4)
↓
ARGMAX → class ID
```

Softmax is not required for the final class decision because $\text{argmax}(\text{logits})$ gives the same class as $\text{argmax}(\text{softmax}(\text{logits}))$.

9. Full Integer Validation

The complete integer network was evaluated on all 412 test samples and compared with the original TensorFlow FP32 model.

Validation item	Result
TensorFlow FP32 accuracy	98.30%
Full integer accuracy	98.30%
Prediction differences	0 / 412

This is the key checkpoint: the integer-only software reference produces exactly the same test predictions as the TensorFlow model.

10. FPGA Weight Header

The quantized parameters were exported for the future Vitis HLS implementation:

```
D:\PROJECTS\64feature_NN\RGB_NN\weights\mlp_weights.h
```

The header contains network dimensions, quantization constants, INT8 weight matrices, and INT32 bias arrays.

11. Current Project Status

Stage 1 is complete. The Python integer reference is validated. The FPGA accelerator itself has not yet been implemented.

12. Next Stage — Vitis HLS

Planned next steps: create the 64→32→16→4 C++ HLS implementation; use INT8 inputs/weights and INT32 accumulators/biases; implement ReLU and hidden-layer requantization; run C simulation against the Python golden reference; synthesize and inspect resources/timing; package the IP; integrate on the PYNQ-Z2; and compare hardware predictions with the golden reference.

Checkpoint Summary

Item	Status
64 engineered RGB-derived features	Complete
Lightweight FP32 MLP	Complete
FP32 test accuracy	98.30%
INT8 feature quantization	Verified
INT8 weights / INT32 biases	Implemented
Full integer MLP	Verified
Integer vs TensorFlow predictions	0 differences
mlp_weights.h	Generated
Vitis HLS accelerator	Next stage